

5. Cadenas de Markov

Este tipo de modelos van de las deterministas a las estocásticas.

Cuando variable Y es estocástica, significa que varios valores, los cuales pueden ser de diferentes tipos.

$$\text{valor}(Y) = \{y_1, \dots, y_n\}$$

Su distribución pueden ser diferentes:

$$\text{val}(Y) = \mathbb{R} \rightarrow Y \sim N(\mu, \sigma^2) \rightarrow Pr(Y = y) = \frac{1}{\sqrt{s\pi\sigma^2}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$

FOTO 5 de marzo

Se tiene un conjunto universo, cada subconjunto de este se le llama un evento.

$$\begin{aligned} Pr : \mathbb{P}(U) &\rightarrow \mathbb{R} \\ Pr(\mathbb{A}) &\geq 0 \\ Pr(\mathbb{U}) &\geq 1 \\ Pr(\emptyset) &= 0 \\ Pr(A \cup B) &= P(\mathbb{A}) + P(\mathbb{B}) - P(\mathbb{A} \cap \mathbb{B}) \end{aligned}$$

FOTO 6 de marzo

$A_1, \dots, A_n$ es una partición si y solo si $\bigcup_{i=1}^n A_i = \mathbb{U}$ y $A_i \cap A_j = \emptyset \forall i, j, i \neq j$.

Una variable aleatoria es una función que puede dar diferentes valores:

$$Y \quad \text{val}(Y) = \{y_1, \dots, y_n\}$$

Por ejemplo, queremos si $Pr(Y = y_1)$. Si no hay dudas podemos decir que $Pr(Y = y_1) = Pr(y_1)$. Esto es verdad para los siguientes índices:

$$Y_0 = y_0, Y_1 = y_1, Y_2 = y_2, \dots, Y_T = y_T \rightarrow T_{0:T}, y_{0:T}$$

A esto se le llaman *series de tiempo*.

$Pr(Y_{0:T}) = P(y_0, y_1, y_2, \dots, y_T)$	Verosimilitud
$Pr(Y_T y_{0:T-1})$	Eotimación
$Pr(Y_{T:T+h} Y_{0:T-1})$	Pronostico (Forecasting)
$Pr(Y_T Y_{0:t+1}, Y_{t+1:T})$	Sauvizado (Smoothing)

NOTA: Los ":" son como los índices de los arreglos de Python. 0:T - 1 va de 0 a T - 1

Si $Pr(Y_T | Y_{0:T-1}) \equiv Pr(Y_T | Y_{T-1})$ entonces Y es un proceso de Markov de 1er orden.

Si tenemos $Pr(Y_T|Y_{T-1}, Y_{T-2})$ podemos crear una variable...

$$Z_T = \begin{bmatrix} Y_T \\ Y_{T-1} \end{bmatrix} \rightarrow Pr(Z_T|Z_{T-1}) \rightarrow \begin{bmatrix} Y_T \\ Y_{T-1} \end{bmatrix} \bigg| \begin{bmatrix} Y_{T-1} \\ Y_{T-2} \end{bmatrix}$$

Otra igualdad que podemos conseguir es:

$$Pr(Y_{0:T}) = Pr(Y_T|Y_{1:T-1}) \cdot Pr(Y_{T-1} : Y_{0:T-2}) \dots Pr(Y_1|Y_0)Pr(Y_0)$$

Como decimos que es una cadena de Markov, esto se convierte a:

$$Pr(Y_{1:T}) = Pr(Y_0) \prod_{t=1}^T Pr(Y_t|Y_{t-1})$$

Imaginate que:

$$\text{Val}(Y) = \{G, P\}$$

$$Pr(Y_t = G|Y_{t-1} = G) = .6$$

$$Pr(Y_t = G|Y_{t-1} = P) = .2$$

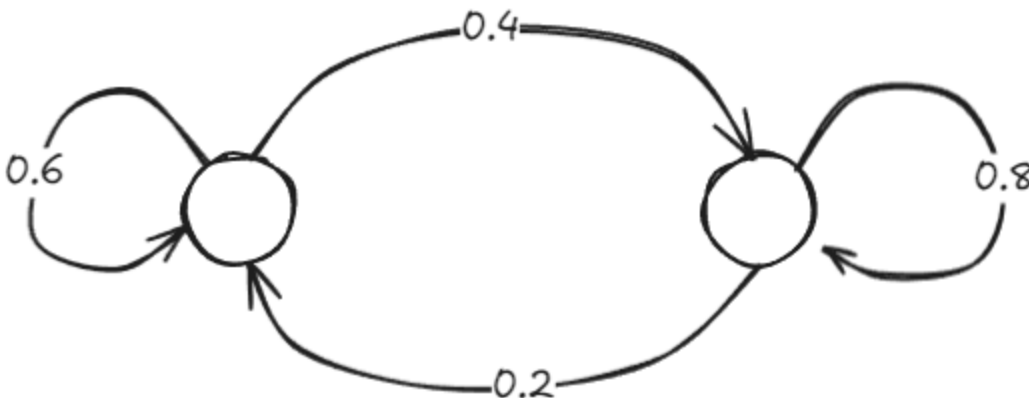
$$Pr(Y_t = P|Y_{t-1} = G) = .4$$

$$Pr(Y_t = P|Y_{t-1} = P) = .8$$

Esto se puede hacer una matriz de transición.

	G	P
G	0.6	0.4
P	0.2	0.8

Y esto subsecuentemente se transforma en un grafo:



```
class MDP: # Markov Decision Process
    self acciones_legales(s):
        return conjunto_acciones

    self sucesor(s,a, ss):
        return Pr(St_1 = ss | St = s At = a) # 1
```

```

self estados():
    return iterableConTodosLosEstados

self recompensa(s,a,ss):
    return numero

self terminal(s):
    return True/False

self.factor_descuento

```

$$1) Pr(S_{t+1} = s' | S_t = s, A_t = a)$$

Los datos que tenemos son:

- Estados: $S = \{S_1, \dots, S_n\}$
Conjunto de estados discretos
- Acciones: $\{a_1, \dots, a_m\}$
- Acciones legales: $A_{legal} : S \rightarrow P(A)$
 $A_e(s)$ es on conjunto de acciones legales en s
- Estado sucesor: $T : S \times A \times S \rightarrow \mathbb{R}$
 $T(s, a, s')$ es la probabilidad de llegar a s' donde s usando la acción a
- Factor de descuento: $r : S \times A \times S \rightarrow \mathbb{R}$
 $r(s, a, s')$ es la recompensa de llegar a s' desde s con la acción a .
- Estado terminal: $S_T \subseteq S$
Conjunto de estados terminales
- $0 \leq \gamma \leq 1$
Factor de descuento

Queremos establecer una política π (estocástica):

$$\pi : S \times A \rightarrow \mathbb{R} \text{ donde } \pi(s, a) = Pr[A_T = a | S_T = s]$$

Para cada $s \in S$ necesitamos una política:

$[R_t | S_T = S]$ sea máxima.

$$R_T = r_T + \gamma r_{t+1} + \gamma^2 r_{t+2} + \dots$$

¿Como lo consigo?

$$\begin{aligned} & : S \rightarrow A \\ & : S \times A \rightarrow \mathbb{R} \end{aligned}$$

¿y si tengo dos políticas π_1 y π_2 , cuál es mejor? Calculamos la función de valor de estado de

$$\pi(s) = \pi[R_T | S_T = s]$$

Sacamos la esperanza:

$$\begin{aligned} R_T &= r_T + r_{T+1} + {}^2r_{T+2} + {}^3r_{T+3} + \dots \\ R_T &= r_T + [r_{T+1} + r_{T+2} + {}^2r_{T+3} + \dots] \\ R_T &= r_T + R_{T+1} \end{aligned}$$

Con esto sacamos una π determinista.

$$\begin{aligned} \pi(s) &= \sum_{s' \in S} T(s, \pi(s), s') [r(s, \pi(s), s') + [R_{T+1} | S_{T+1} \in s']] \\ \pi(s) &= \sum_{s' \in S} T(s, \pi(s), s') [r(s, \pi(s), s') + \pi(s')] \end{aligned}$$

También podemos sacar una π estocástica.

$$\pi(s) = \sum_{a \in A_{\text{legal}}(s)} \pi(s, a) \sum_{s' \in S} T(s, a, s') [r(s, a, s') + \pi(s')]$$

Si $\pi(s) \leq \pi_2(s) \quad \forall s \in S$, entonces $\pi_1 \leq \pi_2$ es mejor la política π_2 .

π_1 y π_2 y hago:

$$\pi_3 \text{ tal que } \pi_3(s) = \begin{cases} \pi_1(s) & \text{si } \pi_1(s) \geq \pi_2(s) \\ \pi_2(s) & \text{si } \pi_2(s) > \pi_1(s) \end{cases}$$

entonces:

$$\pi_1 \leq \pi_3 \text{ y } \pi_2 \leq \pi_3$$

entonces π^* tal que $\pi^*(s) \geq \pi(s) \quad \forall \pi$

Función de valor de Estado-Acción

$$\begin{aligned} \pi(s, a) &= \pi(R_T | S_T = s, A_T = a) \\ \pi(s, a) &= \sum_{s' \in S} T(s, a, s') [r(s, a, s') + \pi(s')] \end{aligned}$$

donde:

$$\pi(s) = \sum_{a \in A(s)} \pi(s, a)$$

Si se aplica en la función de estado-acción se usa, lo siguiente:

$$\begin{aligned} & \sum_{a' \in A(s')} \pi(s', a') \pi(s', a') \\ \pi(s_1) &= \sum_{s' \in S} T(s_1, \pi(s_1), s') [r(s_1, \pi(s_1), s') + \pi(s')] \\ \pi(s_2) &= \sum_{s' \in S} T(s_2, \pi(s_2), s') [r(s_2, \pi(s_2), s') + \pi(s')] \\ & \vdots \\ \pi(s_n) &= \sum_{s' \in S} T(s_n, \pi(s_n), s') [r(s_n, \pi(s_n), s') + \pi(s')] \end{aligned}$$

Toda esta cosa se traduce a:

$$\pi = f(\pi)$$

Este es un problema de punto fijo para encontrar políticas de manera eficiente. En otras palabras, se llama iteración de políticas.

Pseudocódigo:

Inicializar (s) aleatorio $\forall s \in S$

```
def calcula_V(mdp, politica, tolerancia = 1e-6):
    V = {s:0 for s in mdp.estados()}
    for _ in range(1e10):
        delta = 0
        for s in mdp.estados():
            Vp = sum(mdp.sucesor(s, politicas[s], s_p) * [mdp.recompensa(s,
politica[s], s_p + mdp.descuento + V[s_p for s_p in mdp.estados()])]
            delta = max(delta, abs(vp, V[s]))
            V[s] = Vp
            if delta < tolerancia:
                break
    return V
```

Repaso

Los datos que tenemos en un MDP son:

- Estados: $S = \{S_1, \dots, S_n\}$
Conjunto de estados discretos
- Acciones: $\{a_1, \dots, a_m\}$
- Acciones legales: $A_{legal} : S \rightarrow P(A)$
 $A_e(S)$ es on conjunto de acciones legales en s
- Estado sucesor: $T : S \times A \times S \rightarrow \mathbb{R}$
 $T(s, a, s')$ es la probabilidad de llegar a s' donde s usando la acción a
 $[T(s, a, s') = Pr(S_{\pi+1}|S_T = s, A_T = a)]$
- Factor de descuento: $r : S \times A \times S \rightarrow \mathbb{R}$
 $r(s, a, s')$ es la recompensa de llegar a s' desde s con la acción a .
- Estado terminal: $S_T \subseteq S$
Conjunto de estados terminales
- $0 \leq \gamma \leq 1$
Factor de descuento

Queremos establecer una política π (estocástica):

$$\pi : S \times A \rightarrow \mathbb{R} \text{ donde } \pi(s, a) = Pr[A_T = a | S_T = s]$$

¿Cómo calculamos que tan buena es una política? Usando la función del valor de estado.

$$\pi(s) = \pi[R_T | S_T = s]$$

Esto se puede descomponer a:

$$\begin{aligned} R_T &= r_T + r_{T+1} + {}^2r_{T+2} + {}^3r_{T+3} + \dots \\ R_T &= r_T + [r_{T+1} + r_{T+2} + {}^2r_{T+3} + \dots] \\ R_T &= r_T + R_{T+1} \end{aligned}$$

Y terminamos con:

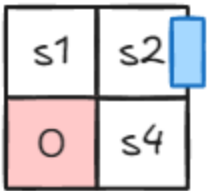
$$\pi(s) = \sum_{a \in A_{\text{legal}}(s)} \pi(s, a) \sum_{s' \in S} T(s, a, s') [r(s, a, s') + \pi(s')]$$

...considerando que iniciamos de un estado inicial:

$$\pi(s) = \pi[R_T | S_T = s]$$

Por ejemplo

Imagina que tienes 4 cuartos y tu punto quiere salir:



Las probabilidades de movimiento son:

- Arriba
- Abajo
- Izquierda
- Derecha

Nuestras recompensas son 1 y -1. Vamos a establecer 2 políticas:

$$\begin{aligned} \pi_1(s, \rightarrow) &= 1 \forall s \\ \pi_2(s, a) &= \frac{1}{4} \forall s, \forall a \end{aligned}$$

Ahora vamos a calcular la probabilidad de irnos hacia la derecha ($\rightarrow$) si estamos en s_1 y queremos quedar en s_1 haciendo uso de $\pi(s)$:

$$\begin{aligned} \pi_1(s_1) &= P(s_1, \rightarrow, s_1)[-1 + \pi_2(s_1)] + P(s_1, \rightarrow, s_2)[1] + P(s_1, \rightarrow, s_3)[-1 + \pi_2(s_3)] \\ &\quad + P(s_1, \rightarrow, s_4)[-1 + \pi_2(s_4)] \end{aligned}$$

Evaluamos

$$\pi_1(s_1) = 0.05[-1 + \pi_2(s_1)] + 0.9 + 0.05[-1 + \pi_2(s_3)]$$

$$\pi_1(s_3) = 0.05[-1 + \pi_2(s_1)] + 0.9[-1 + \pi_2(s_4)] + 0.05[-1 + \pi_2(s_3)]$$

$$\pi_1(s_4) = 0.9[-1 + \pi_2(s_4)] + 0.05[-1 + \pi_2(s_4)] + 0.05[1]$$

No evaluamos $\pi_1(s_2)$ porque s_2 es nuestro estado terminal. Por la misma razón aparece el $[1]$ en las ecuaciones. Todas estas funciones son lineales y, por lo tanto, se puede encontrar una solución respecto a $\pi_1(s_1), \pi_1(s_3), \pi_1(s_4)$.

Si tenemos 2 políticas y el valor de estado en todos los estados es menor o igual a los valores de estados de π_2 , entonces π_2 es mejor que la π_1

$$\pi_1 \text{ es mejor que } \pi_2 \quad \pi_1(s) \leq \pi_2(s) \forall s$$

Si tenemos 2 políticas incomparables, podemos sacar una política para π_3 tal que:

$$\pi_3(s) = \{$$

Para conocer el valor de una política, lo vamos a explicar en código:

$$\pi(s) = \sum_{a \in A_{\text{legal}}(s)} \pi(s, a) \sum_{s' \in S} T(s, a, s') [r(s, a, s') + \pi(s')]$$

```
def iter_politica(mde, politica, max_it = 1e3, tol = 1e-6):
    V = {s:0 for s in mde.S}

    for _ in range(max_it):
        err = 0

        for s in mdp.S:
            vt = sum(politica(s,a) * sum(mdp.T(s,a,ss) +
                [mdp.r(s,a,ss) + mdp.gamma + V[ss] + for ss in mdp.S
                ) for a in mdp.a_legales(s))
            err = max(err, abs(V[s]-vt))
            V[s] = vt
        if err < tol:
            break
    return V
```

Agarramos otra política:

$$\pi(s, a) = \pi[R_T | S_T = s, A_T = a]$$

$$\pi(s, a) = \sum_{s' \in S} T(s, a, s') \left[r(s, a, s') + \sum_{a' \in A_I(s')} \pi(s', a') \pi(s', a') \right]$$

Con esto sacas:

$$\pi(s) = \max_{a \in A_I(s)} \pi(s, a)$$

Lo cual puede reemplazar la parte derecha de la ecuación grande de arriba.

Tenemos dos políticas:

$$\pi \leq \pi^*$$

$$\forall \pi \in \Pi$$

- π^* es la política óptima.
- V^* y Q^* son las funciones de valor óptimas.

$$V^*(s) = \pi^*(s), Q^*(s, a) = \pi^*(s, a)$$

Entre las dos políticas V^* y Q^* podemos decidir cual tiene le mayor valor.

$$\pi(s, a) \begin{cases} 1 & \text{si } a = \arg \max_{a \in A_I(s)} V^*(s, a) \\ 0 & \text{en otro caso} \end{cases}$$

Por ejemplo:

Si tienes $V^*(s, a) = 5$ y $Q^*(s, a) = 7$. Si tienes

Con esto terminamos con la **Ecuación de Optimalidad de Bellman**:

$$Q^*(s, a) = \max_{s' \in S} T(s, a, s') [V^*(s, a, s') + \max_{a' \in A_I(s')} Q^*(s', a')] \\ V^*(s) = \max_{a \in A_I(s)} Q^*(s, a)$$

Esto se traduce a:

```
def iter_valor(MDP, max_iter = 1e5, tol = 1e-6):
    Q = {(s,a): 0 for s in MDP.estados() for a in MDP.acciones_legales(s)}
    for _ in range(max_iter):
        err = 0

        for (s,n) in Q.keys():
            q = sum(MDP.T(s,a,ss) * MDP.r(s,n,ss) + MDP.gamma * max(
                Q[(ss,aa)] for aa in MDP.acciones_legales(ss))
                for ss in MDP.estados())

            err = max(err, abs(Q[(s,a)] - q))
            Q[(s,a)] = q

    if err < tol:
        break
```



```

return Q

def genera_politica_greedy(MDP, Q):
    pi = {}

    for s in MDP.estados():
        pi[s] = max(a for a in MDP.acciones_legales(s), key = lambda
a:Q[(s,a)])

    return pi

```

Recordando. Un Markov Decision Process (MDP) requiere de:

- *Estados.* $S = (S_1, \dots, S_n)$
- *Acciones.* $A = (a_1, \dots, a_m)$
- *Acciones legales.* $A_L : S \rightarrow P(A)$
- *Estados terminales.* $S_T \subseteq S$
- $T : S \times A \times S \rightarrow \mathbb{R} \quad T(s, a, s') = P(S_{t+1} = s' | S_t = s, A_t = a)$
- $r : S \times A \times S \rightarrow \mathbb{R} \quad r(s, a, s')$
- $\gamma \in [0, 1]$

Nota: $T(s, a, s') = P(S_{t+1} = s' | S_t = s, A_t = a)$ también puede ser $\sum_{i \in S} T(s, a, s') = 1$.

Ahora entramos con las políticas:

- $\pi : S \times A \rightarrow \mathbb{R} \quad \pi(s, a) = P(A_t = a | S_t = s) \quad \sum_{a \in A_L(s)} \pi(s, a) = 1$

$$\begin{aligned}
 \pi^* &= \arg \max_{\pi} \pi[R_T] \quad \forall s \in S \\
 \pi^* &= \arg \max_{\pi} [\pi[r_t + \gamma V_{T+1} + (1-\gamma)V_{T+2} + \dots]] \quad \forall s \in S \\
 \pi^* &= \arg \max_{\pi} [\pi[r_T + V_{T+1}]] \quad \forall s \in S
 \end{aligned}$$

$$\pi = \pi[r_T + V_{T+1} | S_T = s] = \sum_{a \in A_L(s)} \pi(s, a) \sum_{s' \in S} T(s, a, s') [r(s, a, s') + \gamma V(s')]$$

$$V^*(s) = \max_{a \in A_L(s)} \sum_{s' \in S} T(s, a, s') [r(s, a, s') + \gamma V^*(s')]$$

$$\pi^*(s, a) = \begin{cases} 1 & \text{si } a = \arg \max_{a \in A_L(s)} \sum_{s' \in S} T(s, a, s') [r(s, a, s') + \gamma V^*(s')] \\ 0 & \text{en otro caso} \end{cases}$$

Podemos hacer un simulador de MDP (DP_{sim}):

- $S = (s_1, \dots, s_n)$
- $A = (a_1, \dots, a_m)$
- $A_L : S \rightarrow P(A)$
- $S_T \subseteq S$
- $s' = \text{sucesor}(s, a)$

- $r : S \times A \times S \rightarrow \mathbb{R} \quad r(s, a, s')$
- $\gamma \in [0, 1]$

¿Como evaluamos nuestra política π con DP_{sim} ?

$$\pi(s) = \sum_{a \in A_L(s)} \pi(s, a) \sum_{s' \in S} T(s, a, s') [r(s, a, s') + \gamma \pi(s')]$$

Asumimos que conocemos $\pi(s)$ y estoy en el estado S , escojo a a partir de π . Aplico a a DP_{sim} y obtengo s' y r .

- $s' = \text{sucesor}(s, a)$
- $r = r(s, a, s')$
entonces $\pi_{t+1}(s) = r + \gamma \pi(s')$.

$$\pi(s) = \pi + [\pi_{t+1}(s) - \pi(s)]$$

A lo que esta en llaves se le llama *Diferencia Temporal (TD)*

Considera que política = π

```
if politica == "Determinista":
    a = politica(s)
if politica == "Estocastico":
    umbral = random.random()
    nivel = 0

    for a in acciones_legales(s):
        nivel += politica(s,a)

        if umbral <= nivel:
            return a
    raise ValueError("-")
```

```
class MDPsim:
    def __init__(self, s, gamma):
        self.s = s
        self.gamma = gamma
    def acciones_legales(self, s):
        return iterable con acciones legales en s
    def sucesor(self, s, a):
        return ss # ss es s'
    def recompensa(self, s, a, ss):
        return r
    def terminal(self, s):
        return True/False
```

```
def estado_inicial(self):
    return estado
```

Ahora tenemos que hacer una política:

```
class Politica:
    def __init__(self, MDPsim):
        self.MDPsim = MDPsim
    def __item__(self, s)
        return self.pi[s]
```

Considerando que:

$$\pi(s) = \sum_{a \in A_L(s)} \pi(s, a) \sum_{s' \in S} T(s, a, s') [r(s, a, s') + \pi(s')]$$

Tenemos un $\pi(s)$ para un estado s . Aplicamos:

- $a = \text{Política}(s)$
- $s' = \text{DPsim.sucesor}(s, a)$
- $r = \text{DPsim.recompensa}(s, a, s')$
- $= r + \text{DPsim} \cdot \pi(s')$

Y terminamos con Diferencia Temporal (TD)

$$\pi(s) = \pi + [\pi_{t+1}(s) - \pi(s)]$$

Vamos a programarlo:

```
def TD_0(mdl, pi, alpha, max_episodios):
    V = {o for s in mdl.S}

    for _ in range(max_episodios):
        err = 0
        s = mdl.estado_inicial()

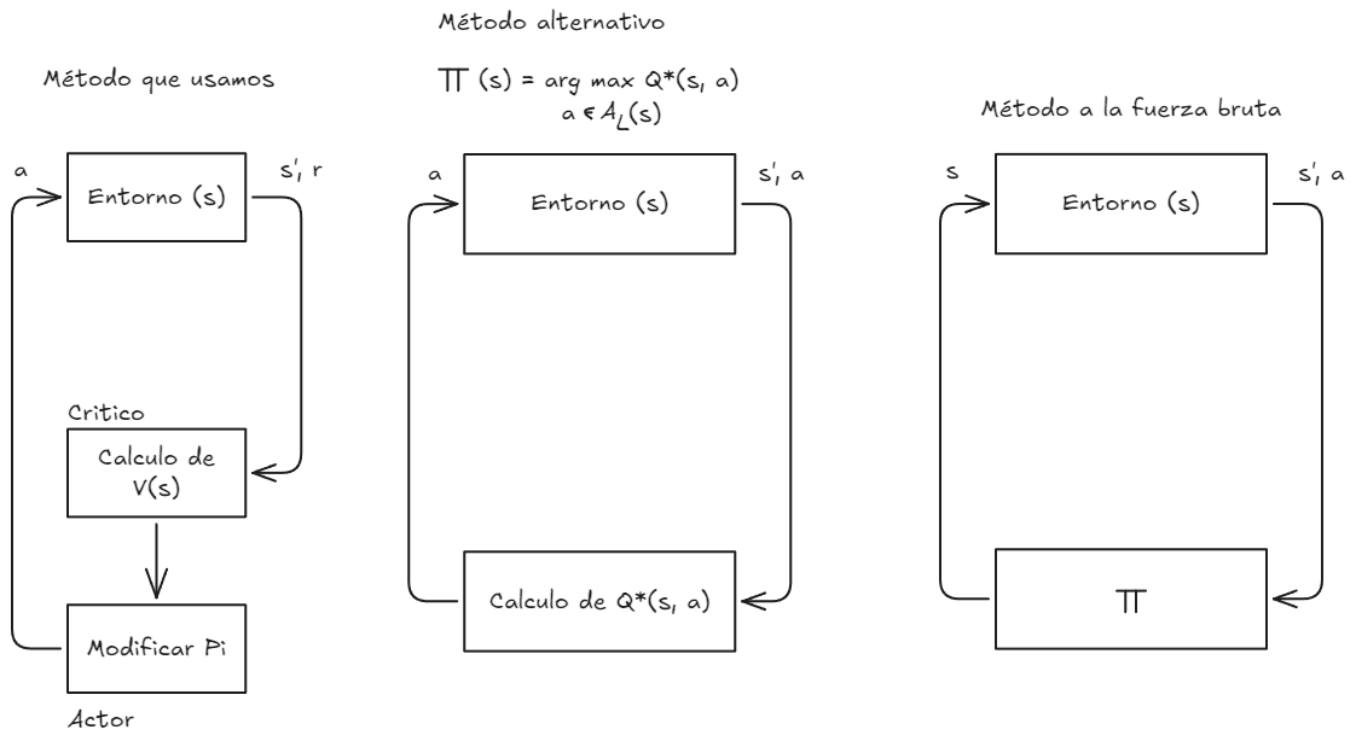
        for _ in range(max_it):
            a = pi(s)
            ss = mdl.sucesor(s, a)
            r = mdl.recompensa(s, a, ss)
            v = v[s] - alpha * (r - mdl.gamma * v[ss] - v[s])
            err = max(err, abs(v - V[s]))
            v[s], s = v, ss

        if mdl.terminal(s):
            break
```

```

    if err < tol:
        break
    return v

```



Usamos (s, a)

$$\pi(s, a) = \sum_{s' \in S} T(s, a, s') \left[r(s, a, s') + \max_{a' \in A_L(s')} \pi(s', a')(s', a') \right]$$

$$Q^*(s, a) = \sum_{s' \in S} T(s, a, s') [r(s, a, s') + \max_{a' \in A_L} Q^*(s', a')]$$

Un ejemplo de uso es:

- Tengo (s, a)
- Aplico:
- $s' = mdl.sucesor(s, a)$
- $r = mdl.recompensa(s, a, s')$

$$\begin{cases} a' = \arg \max_{a'} Q(s', a') \\ a = política() \end{cases}$$

- $q = r + mdl.Q(s', a')$

$$Q(s, a) = Q(s, a) + (q - Q(s, a))$$

Jugando Blackjack

Vamos a hacer un estado de ejemplo

$$S = (n_1, n_2, a_1, a_2, d, ci, n_3, a_3, d_2)$$

Nuestra intención es encontrar una política óptima usando programación dinámica.

En el blackjack solamente nos fijamos en el valor de la carta y no consideramos color.
Entonces....

<i>A</i>	1	2	3	4	5	6	7	8	9	10	—	—	<i>R</i>
—	—	—	—	—	—	—	—	—	—	—	—	—	—
$\frac{1}{14}$					$\frac{1}{14}$					$\frac{1}{14}$		$\frac{3}{14}$	

El factor de descuento no va a ser 0 porque no tenemos nada que olvidar.